

Get it working

To have this application work in a browser, use a simple Text Area component that renders basic HTML

Flash support

Flash Builder 4 has recently been updated to use the new Flex 4.1 SDK that adds support for Flash Player 10.1



Creating a simple RSS feed reader with Flash Builder 4

Most websites publish feeds of its content. Using feeds allows one to have a central place to receive all updates from web sites of their interest. By pulling such content into a desktop client, they get desktop notifications and can archive the feed contents if they want, and control how they consume the content. We'll explore how to create our own feed reader here.

We will write this application for Adobe AIR, not the Flash Player for the web. Adobe AIR includes the WebKit engine – the same engine that powers Google Chrome and Apple's Safari – so we can have an entire browser in our application. Including the browser component is simple – add a tag in our MXML code, and controlling it from your code is quite simple as well. We will make use of this feature displaying the feed content in a rich format. Since the WebKit engine isn't included in Flash Player, converting this application for use in a browser will not be a simple task, and will not be powerful enough.

First of all we will create the UI layout for the application, then we will add the code to make the application actually load the feed, display it and change the displayed article with changes in the list selection. Finally we will extend the application just a bit to give the user the option to view either the article summary / extract or the entire article at the original location.

Part 1: The UI

The UI of the application is quite simple. We have a text input box where the user can input the feed location, right next to which, is a button labelled "Load" for loading the feed into the application.

Below that is a List. The list will display the titles of all the items in the feed. Clicking on any item in the feed will display the extract in the area below.

Let us break down the UI. First we have a TextInput component placed in a horizontal layout with the "Load" button:

```
<s:HGroup width="100%">
  <s:TextInput id="feedUrl"
width="100%" />
  <s:Button id="loadFeed"
label="Load" />
</s:HGroup>
```

Notice that we have used the short form HGroup instead of creating a Group and setting its layout to Horizontal-Layout.

This composite of the TextInput for the feed URL and the "Load" button is then vertically aligned with the List and the WebKit Component. To do this we can set the layout of the entire application to VerticalLayout, and add the list and WebKit component both in the root of the application. The WebKit component can be included by using the HTML component.

Your final code should look something like the following:

```
<?xml version="1.0"
encoding="utf-8"?>
<s:WindowedApplication
xmlns:fx="http://ns.adobe.com/
mxml/2009"
xmlns:s="library://ns.adobe.
com/flex/spark"
xmlns:mx="library://ns.adobe.
com/flex/mx"

width="800" height="600"
>
<s:layout>
<s:VerticalLayout />
</s:layout>
<s:HGroup width="100%">
  <s:TextInput id="feedUrl"
width="100%" />
  <s:Button id="loadFeed"
label="Load" />
</s:HGroup>
<s:List id="feedItems"
width="100%" height="100%" />
```

```
<mx:HTML id="article"
width="100%" height="100%" />
</s:WindowedApplication>
```

Part 2: The functionality

Now we need to add some event handlers for the UI elements. We need to have the application start loading the URL when we click the "Load" button, so let's start there. Add a click handler for the "Load" button, as follows:

```
protected function loadFeed_
clickHandler(event:MouseEvent):
void
{
  var ul:URLLoader =
new URLLoader();
  ul.addEventListener
(Event.COMPLETE, onFeed
Loaded);
  ul.load(new
URLRequest(feedUrl.text));
}
```

This function is doing the following:

1. var ul:URLLoader = new URLLoader();
This created a URLLoader, which will load the contents of our feed
2. ul.addEventListener(Event.COMPLETE, onFeedLoaded);
This adding an event handler to our URLLoader which will be called when the "COMPLETE" event occurs. This event occurs when it has completed loading the whatever URL is assigned to it.
3. ul.load(new URLRequest(feedUrl.text));
This line of code tells the URLLoader to actually start loading the URL we specify. We provide the URL to the URLLoader in the form of a URLRequest object which is assigned the URL of the feed the user entered in the text box (feedURL.text).

We will also create an object to store the data retrieved from the feed. Let us